LAB 1: BUFFER

INFOE 3

Group Members:

Joy Melisa S. Segotier

Jorlyn Shane M. Claveria

Maria Theresa N. Reyes

1. When you successfully used the payload to overwrite the secret key variable, which pillar of the C.I.A. triad did you primarily violate? Explain why.
   Answer:
   The CIA pillar that was primarily violated is INTEGRITY because you modified the secret key without authorization. Even though you didn't "log in," you changed the data stored in the system.

2. What specific vulnerability category did you exploit when you entered a string longer than 8 characters?
   Answer:
   The specific vulnerability that got exploited when entered a string longer than 8 characters is a Buffer Overflow (specifically a Stack-based overflow). It falls under a lack of Bounds Control.

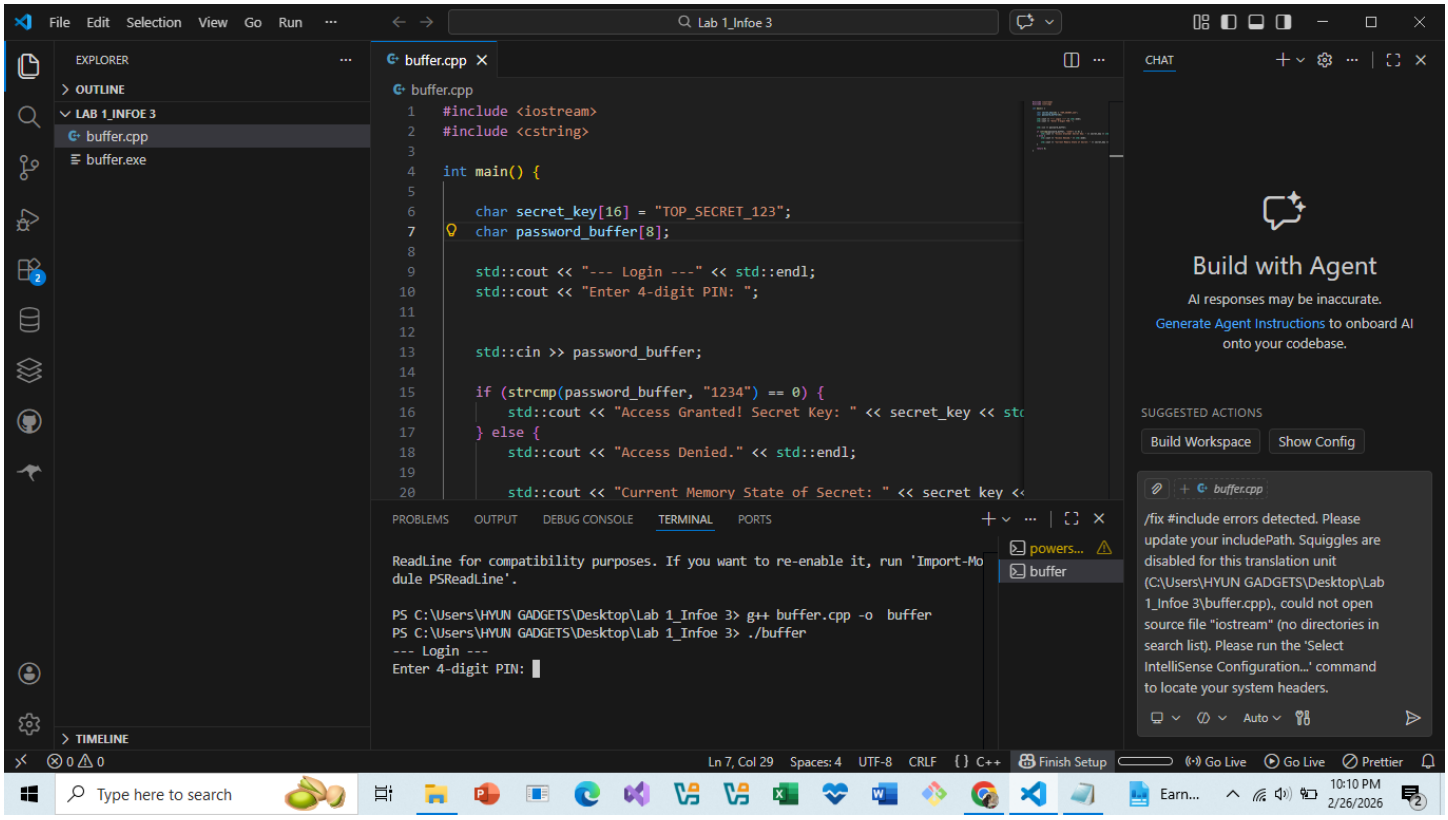
3. Why is your solution (#4 The Fix) more effective than simply "asking users to be honest"?
   Answer:
   Relying on user honesty is a "soft" control. The code fix is a Technical or Logical Control that physically prevents the memory from being overwritten, regardless of what the user types.

4. Imagine this was running the control system for a local power grid.
   * Who might want to exploit this vulnerability?
     Answer:
     State-sponsored actors, eco-terrorists, or black-hat hackers.

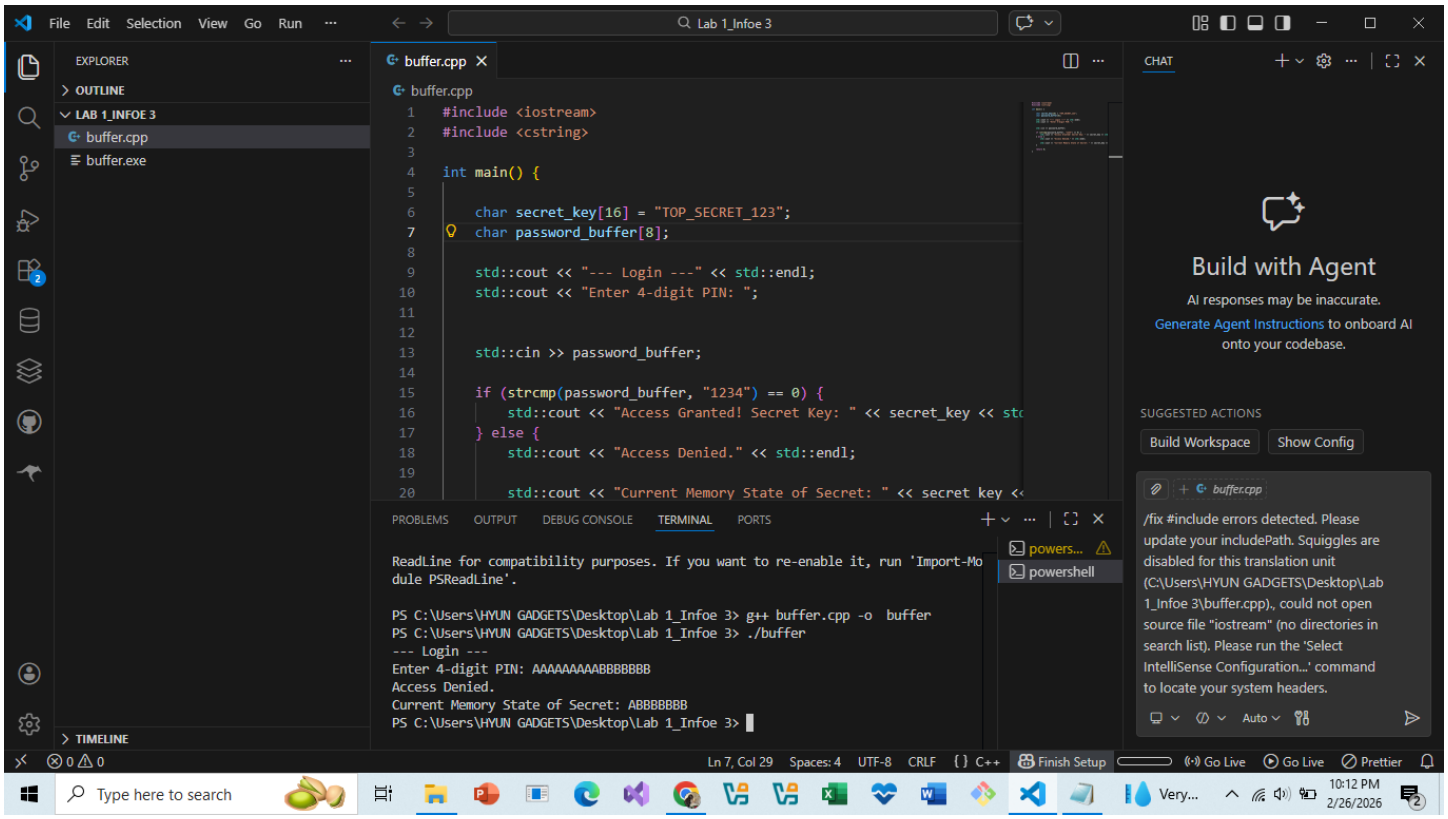
   * What would be the real-world consequence if the secret key (controlling the power switch) was overwritten by a pro cyber-attacker?
     Answer:

If the key controls a switch, overwriting it could cause a permanent lockout (Denial of Service) or allow an attacker to shut down electricity to a city, potentially leading to hospital failures or infrastructure collapse.

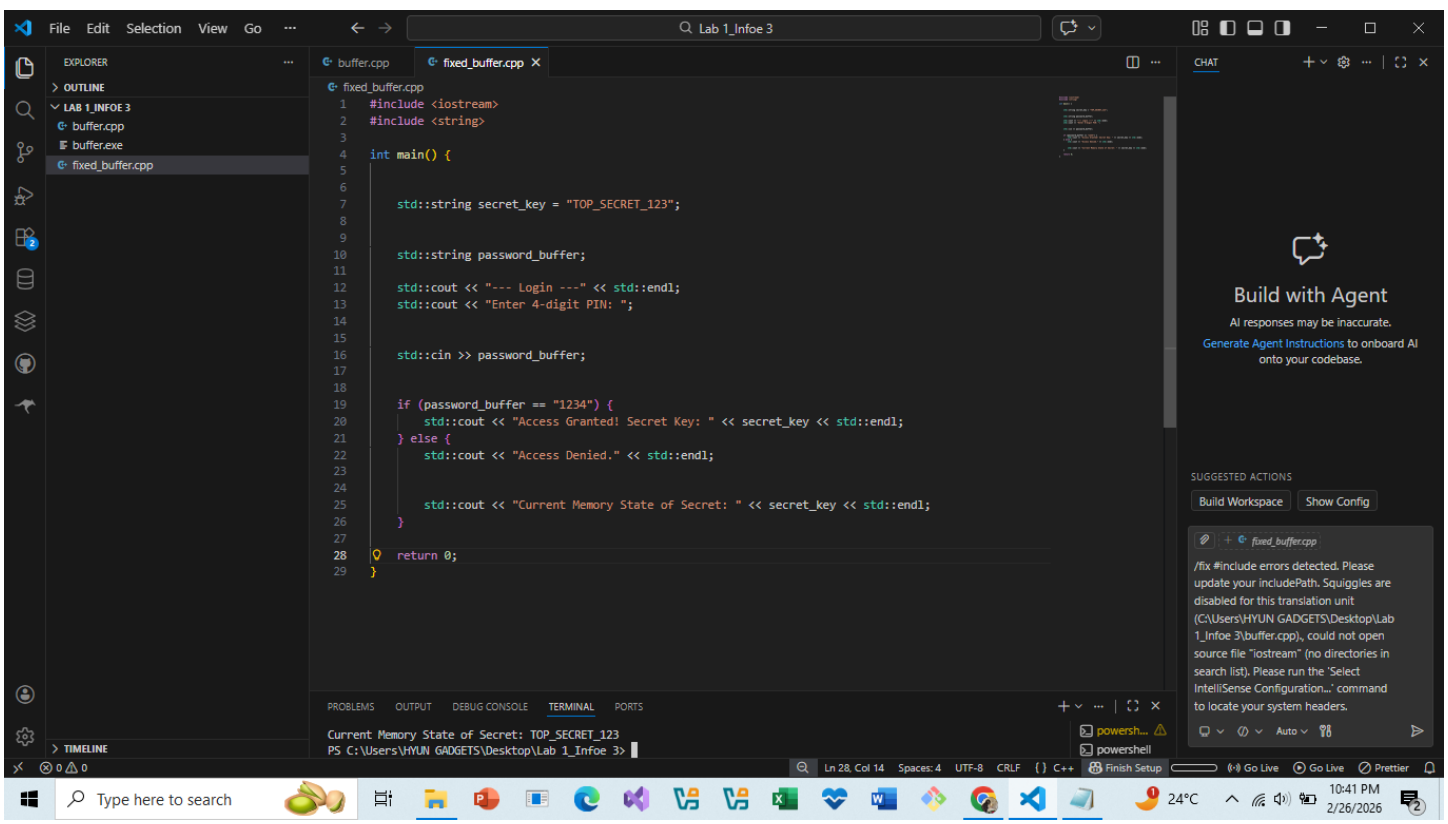
SCREENSHOTS:



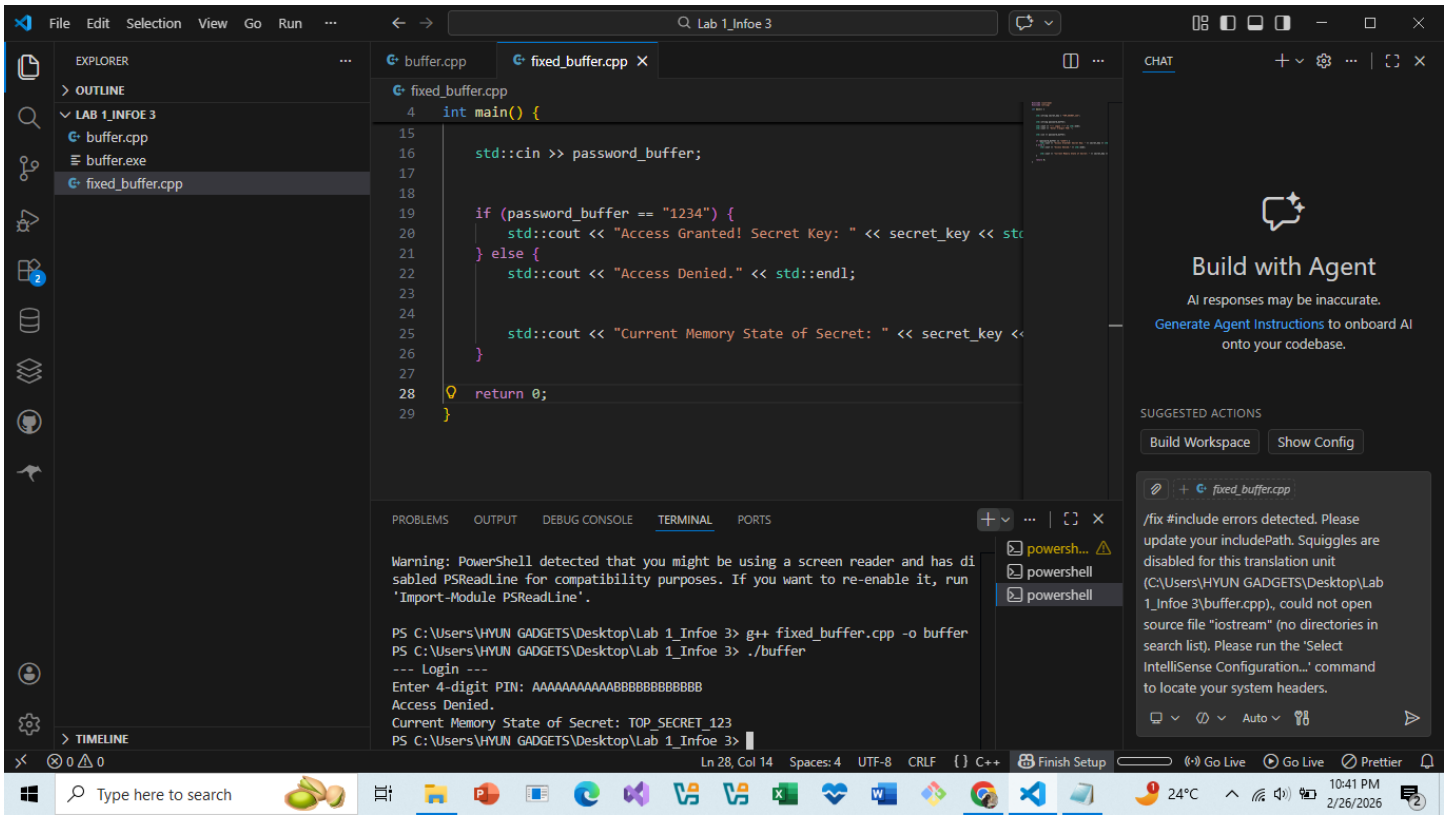
This screenshot shows the initial setup of the buffer.cpp program in VS Code.



This image demonstrates the "Buffer Overflow" vulnerability in action.



This screenshot showcases the corrected version of the program, fixed_buffer.cpp, which uses memory-safe practices.



This image confirms that the fix is effective against the previous attack.